DEDICATED TO INNOVATION

www.bacancy.com

BACANCY®

Customer Satisfaction Is Our Highest Priority

Employee Matters

In this session

What we are learning in this session

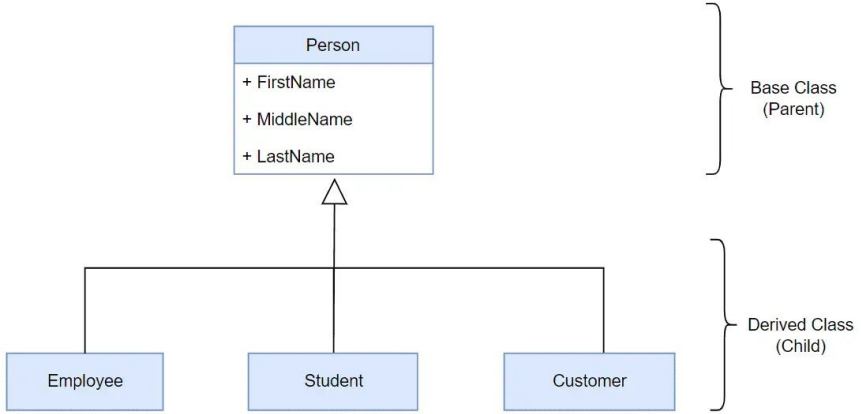
- Inheritance
- Polymorphism
- Abstraction

Let’s Start

Inheritance

- In object-oriented programming, inheritance is another type of relationship between classes. Inheritance is a mechanism of reusing the functionalities of one class into another related class.
- Inheritance is referred to as "is a" relationship. In the real world example, a customer is a person. In the same way, a student is a person and an employee is also a person. They all have some common things, for example, they all have a first name, middle name, and last name.
- To translate this into object-oriented programming, we can create the Person class with first name, middle name, and last name properties and inherit the Customer, Student, and Employee classes from the Person class. That way we don't need to create the same properties in all classes and avoid the violation of the DRY (Do not Repeat Yourself) principle.
- Note that the inheritance can only be used with related classes where they should have some common behaviors and perfectly substitutable.

Inheritance



- In C#, use the : symbol to inherit a class from another class. For example, the following Employee class inherits from the Person class in C#.

- **Example: Class Inheritance in C#**

```
class Person
{
   public string FirstName { get; set; }
   public string LastName { get; set; }

   public string GetFullName(){
      return FirstName + " " + LastName;
   }
}

class Employee : Person
{
   public int EmployeeId { get; set; }
   public string CompanyName { get; set; }
}
```

Inheritance

- In the above example, the Person class is called the base class or the parent class, and the Employee class is called the derived class or the child class.
- The Employee class inherits from the Person class and so it automatically acquires all the public members of the Person class. It means even if the Employee class does not include FirstName, LastName properties and GetFullName() method, an object of the Employee class will have all the properties and methods of the Person class along with its own members.

Note: C# does not allow a class to inherit multiple classes. A class can only achieve multiple inheritance through interfaces.

Inheritance

- **Constructors:**

  Creating an object of the derived class will first call the constructor of the base class and then the derived class. If there are multiple levels of inheritance then the constructor of the first base class will be called and then the second base class and so on.

- **Object Initialization:**

  You can create an instance of the derived class and assign it to a variable of the base class or derived class. The instance's properties and methods are depending on the type of variable it is assigned to. Here, a type can be a class or an interface, or an abstract class.

- **The following table list supported members based on a variable type and instance type.**

Instance variable	Instance Type	Instance Members of
Base type	Base type	Base type
Base type	Derived type	Base type
Derived type	Derived type	Base and derived type

Inheritance

```
class Person
{
    public string FirstName { get; set; }
    public string LastName { get; set; }

    public string GetFullName()
    {
        return FirstName + " " + LastName;
    }
}

class Employee : Person
{
    public int EmployeeId { get; set; }
    public string CompanyName { get; set; }

    public void DisplayEmployeeDetails()
    {
        Console.WriteLine($"Employee ID: {EmployeeId}");
        Console.WriteLine($"Name: {GetFullName()}");
        Console.WriteLine($"Company: {CompanyName}");
    }
}
```

```
static void Main(string[] args)
    {
        // Case 1: Base type variable, Base type instance
        Person person = new Person();
        person.FirstName = "John";
        person.LastName = "Doe";
        Console.WriteLine("Case 1: Base type variable, Base type instance");
        Console.WriteLine($"Full Name: {person.GetFullName()}");
        // person.EmployeeId = 101; // Error: Employee-specific members not accessible
        Console.WriteLine();

        // Case 2: Base type variable, Derived type instance
        Person personAsEmployee = new Employee();
        personAsEmployee.FirstName = "Jane";
        personAsEmployee.LastName = "Smith";
        Console.WriteLine("Case 2: Base type variable, Derived type instance");
        Console.WriteLine($"Full Name: {personAsEmployee.GetFullName()}");
        // personAsEmployee.EmployeeId = 102; // Error: Employee-specific members not accessible
        Console.WriteLine();

        // Case 3: Derived type variable, Derived type instance
        Employee employee = new Employee();
        employee.FirstName = "Alice";
        employee.LastName = "Brown";
        employee.EmployeeId = 202;
        employee.CompanyName = "Tech Corp";
        Console.WriteLine("Case 3: Derived type variable, Derived type instance");
        Console.WriteLine($"Full Name: {employee.GetFullName()}");
        Console.WriteLine($"Employee ID: {employee.EmployeeId}");
        Console.WriteLine($"Company Name: {employee.CompanyName}");
        employee.DisplayEmployeeDetails();
        Console.WriteLine();

    }
```

Inheritance

- **Important Points:**
  - A class can inherit a single class only. It cannot inherit from multiple classes.
  - A class cannot inherit from a struct.
  - A class can inherit (implement) one or more interfaces.
  - A Struct can inherit from one or more interfaces. However, it cannot inherit from another struct or class.
  - An interface can inherit from one or more interfaces but cannot inherit from a class or a struct.
  - Constructors or destructors cannot be inherited.

Polymorphism

- Polymorphism is a Greek word that means multiple forms or shapes. You can use polymorphism if you want to have multiple forms of one or more methods of a class with the same name.
- **In C#, polymorphism can be achieved in two ways:**
  - Compile-time Polymorphism
  - Run-time Polymorphism
- **Compile-time Polymorphism (Method Overloading):**
  - Compile-time polymorphism is also known as method overloading. C# allows us to define more than one method with the same name but with different signatures. This is called method overloading.
  - Method overloading is also known as early binding or static binding because which method to call is decided at compile time, early than the runtime.
- **Rules for Method Overloading:**
  - Method names should be the same but method signatures must be different. Either the number of parameters, type of parameters, or order of parameters must be different.
  - The return type of the methods does not play any role in the method overloading.
  - Optional Parameters take precedence over implicit type conversion when deciding which method definition to bind.

Method Overloading

```
class MathOperations
{
    // Method with two int parameters
    public void Add(int a, int b)
    {
        Console.WriteLine("Sum of integers: " + (a + b));
    }

    // Overloaded method with three int parameters
    public void Add(int a, int b, int c)
    {
        Console.WriteLine("Sum of three integers: " + (a + b + c));
    }

    // Overloaded method with double parameters
    public void Add(double a, double b)
    {
        Console.WriteLine("Sum of doubles: " + (a + b));
    }
}
```

```
static void Main()
    {
        MathOperations math = new MathOperations();

        math.Add(5, 10);       // Calls Add(int, int)
        math.Add(5, 10, 15);   // Calls Add(int, int, int)
        math.Add(5.5, 2.5);    // Calls Add(double, double)
    }
```

Method Overriding

- Run-time polymorphism is also known as inheritance-based polymorphism or method overriding.
- Inheritance allows you to inherit a base class into a derived class and all the public members of the base class automatically become members of the derived class. However, you can redefine the base class's member in the derived class to provide a different implementation than the base class. This is called method overriding that also known as runtime polymorphism.
- In C#, by default, all the members of a class are sealed and cannot be redefined in the derived class. Use the virtual keyword with a member of the base class to make it overridable, and use the override keyword in the derived class to indicate that this member of the base class is being redefined in the derived class

Method Overriding

```
class Person
{
    public virtual void Greet() // Virtual method for overriding
    {
        Console.WriteLine("Hello from Person");
    }
}

class Employee : Person
{
    public override void Greet() // Overriding the base class method
    {
        Console.WriteLine("Hello from Employee");
    }
}
```

```
static void Main(string[] args)
    {
        // Case 1: Base class reference, Base class object
        Person p1 = new Person();
        p1.Greet(); // Calls Person's Greet()

        // Case 2: Base class reference, Derived class object (Upcasting)
        Person p2 = new Employee();
        p2.Greet(); // Calls Employee's overridden Greet() due to polymorphism

        // Case 3: Derived class reference, Derived class object
        Employee emp = new Employee();
        emp.Greet(); // Calls Employee's Greet()

    }
```

Method Overriding

- As you learned in the previous chapter the C# compiler decides which methods to call at the compile time in the compile-time polymorphism. In the run time polymorphism, it will be decided at run time depending upon the type of an object.
- To understand why method overriding is called the runtime polymorphism, look at the given example.
- In the above example, the Display() method takes a parameter of the Person type. Thus, you can pass an object of the Person type or the Employee type when you call the Display() method. The Display() method does not know the type of parameter you passed at compile time. It can be anything at runtime. That's why method overriding is called runtime polymorphism.

```
class Program
{
    // Static method that accepts a Person type but calls the actual object's method
    public static void Display(Person person)
    {
        person.Greet(); // Calls the overridden method at runtime
    }

    static void Main(string[] args)
    {
        Person p = new Person();
        Employee e = new Employee();
        Person pe = new Employee(); // Upcasting: Person reference, Employee object

        Console.WriteLine("Passing Person instance:");
        Display(p); // Calls Person.Greet()

        Console.WriteLine("\nPassing Employee instance:");
        Display(e); // Calls Employee.Greet()

        Console.WriteLine("\nPassing Employee instance as Person type:");
        Display(pe); // Calls Employee.Greet() due to runtime polymorphism
    }
}
```

Runtime Polymorphism (Method Overriding)

Rules for Overriding:

- A method, property, indexer, or event can be overridden in the derived class.
- Static methods cannot be overridden.
- Must use virtual keyword in the base class methods to indicate that the methods can be overridden.
- Must use the override keyword in the derived class to override the base class method.

Virtual Methods

- In C#, a virtual method is a method that can be overridden in a derived class.
- When a method is declared as virtual in a base class, it allows a derived class to provide its own implementation of the method.
- To declare a method as virtual in C#, the "virtual" keyword is used in the method declaration in the base class.
- In the derived class, the method can be overridden by using the "override" keyword in the method declaration. For example.
- Using virtual methods can be useful in situations where you want to provide a base implementation in a base class, but allow derived classes to modify or extend the behavior of that method.
- We can't use the virtual modifier with the static, abstract, private, or override modifiers.

Virtual Methods

```
class Animal
{
    // Virtual method - Can be overridden in derived classes
    public virtual void MakeSound()
    {
        Console.WriteLine("Animal makes a sound");
    }

    // Non-virtual method - Cannot be overridden
    public void Sleep()
    {
        Console.WriteLine("Animal is sleeping");
    }
}

class Dog : Animal
{
    // Overriding the virtual method
    public override void MakeSound()
    {
        Console.WriteLine("Dog barks: Woof Woof!");
    }
}

class Cat : Animal
{
    // Not overriding MakeSound(), so it uses the base class version
}
```

```
static void Main()
    {
        Animal myAnimal = new Animal();
        myAnimal.MakeSound();  // Calls Animal's MakeSound()
        myAnimal.Sleep();      // Calls Animal's Sleep()

        Animal myDog = new Dog();
        myDog.MakeSound();     // Calls Dog's overridden MakeSound()
        myDog.Sleep();         // Calls Animal's Sleep() (non-virtual)

        Animal myCat = new Cat();
        myCat.MakeSound();     // Calls Animal's MakeSound() (not overridden)
        myCat.Sleep();         // Calls Animal's Sleep() (non-virtual)
    }
```

Abstract class and methods

- An abstract class is a class that cannot be instantiated. It is designed to be a base class and must be inherited by other classes. It can have abstract methods (without implementation) and concrete methods (with implementation).
- Abstract classes and methods are declared using the abstract keyword.
- Abstract classes cannot be instantiated directly, but they can be used as base classes for other classes that derive from them.
- A method without the body is known as the Abstract Method. What the method contains is only the declaration of the method.
- Who will Provide the Implementation of Abstract Methods? Answer is child class
- An abstract class can contain both abstract and non-abstract methods. If a child class of an abstract class wants to consume any non-abstract methods of its parent, it should implement all abstract methods.

Abstract class and methods

```
abstract class Animal
{
    public abstract void Speak(); // Abstract method (must be implemented)
}

class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Dog barks: Woof Woof!");
    }
}
```

```
static void Main()
    {
        // Using abstract class reference (Polymorphism)
        Animal animalRef = new Dog();
        animalRef.Speak(); // Calls Dog's Speak() method

        // Using derived class object directly
        Dog myDog = new Dog();
        myDog.Speak(); // Calls Dog's Speak() method
    }
```

Interface

- The Interface in C# is a Fully Unimplemented Class used for declaring a set of operations/methods of an object.
- An interface is a contract that defines what a class should do, but not how it does it. It only contains method declarations (without implementation), and any class that implements the interface must provide implementations for all its methods.
- By default, every member of an interface is abstract, so we aren't required to use the abstract modifier on it again, just like we do in the case of an abstract class.
- We cannot declare fields/variables, constructors, and destructors in an interface in C#.
- C# does not support multiple inheritance for classes (a class cannot inherit from multiple classes) but supports multiple interface implementation. If you need a class to inherit behavior or structure from multiple sources, you can use interfaces to achieve this. This allows you to share code among different classes without the complexities of multiple-class inheritance.

Interface

```
interface IPayment
{
    void MakePayment(double amount); // Method declaration (no implementation)
}

class CreditCardPayment : IPayment
{
    public void MakePayment(double amount)
    {
        Console.WriteLine($"Payment of {amount} made using Credit Card.");
    }
}

class PayPalPayment : IPayment
{
    public void MakePayment(double amount)
    {
        Console.WriteLine($"Payment of {amount} made using PayPal.");
    }
}
```

```
static void Main()
    {
        IPayment payment;

        // Using Credit Card Payment
        payment = new CreditCardPayment();
        payment.MakePayment(500.75);

        // Using PayPal Payment
        payment = new PayPalPayment();
        payment.MakePayment(300.50);
    }
```

Difference between Abstract Class and Interface

Abstract Class	Interface
It contains both declaration and implementation parts.	It contains only the declaration of methods, properties, events or indexes. Since C# 8, default implementations can also be included in interfaces
Multiple inheritance is not achieved by abstract class.	Multiple inheritance is achieved by interface.
It contain constructor.	It does not contain constructor.
It can contain static members.	It does not contain static members.
It can contain different types of access modifiers like public, private, protected etc.	It only contains public access modifier because everything in the interface is public.
The performance of an abstract class is fast.	The performance of interface is slow because it requires time to search actual method in the corresponding class.

Difference between Abstract Class and Interface

Abstract Class	Interface
It is used to implement the core identity of class.	It is used to implement peripheral abilities of class.
A class can only use one abstract class.	A class can use multiple interface.
If many implementations are of the same kind and use common behavior, then it is superior to use abstract class.	If many implementations only share methods, then it is superior to use Interface.
Abstract class can contain methods, fields, constants, etc.	Interface can only contains methods, properties, indexers, events.
The keyword ":" can be used for implementing the Abstract class.	The keyword ":" and "," can be used for implementing the Interface.
It can be fully, partially or not implemented.	It should be fully implemented.

Thank you